



UNIVERSIDAD DE
COSTA RICA
Sede del Atlántico

RP Recinto de
Paraíso

UNIVERSIDAD DE COSTA RICA
SEDE DEL ATLÁNTICO - RECINTO PARAÍSO
CARRERA DE INFORMÁTICA EMPRESARIAL
CURSO: IF0009 - Desarrollo de Software IV
PROFESOR: Mag. Jonathan Granados C.
SEMESTRE: II-2026

Cheatsheet Completo: JavaScript Moderno (ES6+, DOM, Asincronía y Programación Funcional)

Resumen

Esta guía rápida tipo *Cheatsheet* proporciona una referencia exhaustiva del lenguaje **JavaScript** (ECMAScript 2015 / ES6+), abarcando sintaxis moderna, manipulación del DOM, programación funcional de arreglos, manejo asíncrono con Promesas y `async/await`, y patrones clave para el desarrollo web front-end.

1. Declaración de Variables y Ámbito (Scope)

```
// Scope de Bloque ({} ) - Inmutabilidad por referencia (Recomendado por defecto)
const PI = 3.14159;
const usuario = { nombre: 'Ana' };
usuario.nombre = 'Ana Maria'; // Permitido (mutacion interna)

// Scope de Bloque ({} ) - Variable reassignable
let contador = 0;
contador += 1;

// EVITAR: Scope de Funcion y Hoisting propenso a errores
var variableAntigua = 'Obsoleta';
```

2. Operadores Modernos (ES6+)

```
const usuario = {
  id: 1,
  perfil: {
    nombre: 'Carlos',
    contacto: { email: 'carlos@ucr.ac.cr' }
  }
};

// 1. Encadenamiento Opcional (Optional Chaining ?.)
const telefono = usuario?.perfil?.contacto?.telefono; // undefined (sin error)

// 2. Operador de Coalescencia Nula (Nullish Coalescing ??)
// Evalua a la derecha SOLO si la izquierda es null o undefined (difiere de ||)
const nombreMostrado = usuario?.perfil?.nombre ?? 'Usuario Anonimo';
const cantidad = 0 ?? 10; // Retorna 0 (0 no es null/undefined)

// 3. Desestructuracion (Destructuring) de Objetos y Arrays
const { nombre } = usuario.perfil;
const [primero, segundo, ...resto] = [10, 20, 30, 40, 50];

// 4. Operador Spread / Rest (...)
const nuevoUsuario = { ...usuario, activo: true };
const combinacionArrays = [...resto, 60, 70];
```

3. Funciones y Expresiones Flecha (Arrow Functions)

```
// Funcion declarada tradicional (Soporta hoisting)
function sumar(a, b) {
  return a + b;
}

// Funcion flecha (Sintaxis concisa, binding de 'this' léxico)
const multiplicar = (a, b) => a * b;

// Retorno implícito de objeto (envolver en paréntesis)
const crearPersona = (nombre, edad) => ({ nombre, edad });

// Closures (Funciones que recuerdan su entorno léxico)
function crearContador() {
  let cuenta = 0;
  return () => ++cuenta;
}
const incrementar = crearContador();
console.log(incrementar()); // 1
console.log(incrementar()); // 2
```

4. Programación Funcional con Arrays

```
const productos = [
  { id: 1, nombre: 'Teclado', precio: 25.0, categoria: 'Perifericos' },
  { id: 2, nombre: 'Mouse', precio: 15.0, categoria: 'Perifericos' },
  { id: 3, nombre: 'Monitor', precio: 200.0, categoria: 'Pantallas' }
];

// 1. map: Transforma cada elemento (Retorna nuevo arreglo)
const nombres = productos.map(p => p.nombre.toUpperCase());

// 2. filter: Filtra elementos que cumplen una condicion
const baratos = productos.filter(p => p.precio < 30.0);

// 3. reduce: Acumula elementos a un único valor (ej. suma de precios)
const totalInventario = productos.reduce((acc, p) => acc + p.precio, 0);

// 4. find / findIndex / some / every
```

```
const monitor = productos.find(p => p.id === 3);
const hayCaros = productos.some(p => p.precio > 100.0);
const todosPositivos = productos.every(p => p.precio > 0);
```

5. Manipulación del DOM y Eventos

```
// Selección de elementos en el DOM
const botonSubmit = document.querySelector('#btnEnviar');
const items = document.querySelectorAll('.item-lista');

// Crear y modificar elementos
const nuevoDiv = document.createElement('div');
nuevoDiv.textContent = 'Mensaje guardado con éxito';
nuevoDiv.classList.add('alerta', 'alerta-éxito');

// Manejo de Eventos y Delegación de Eventos
document.body.appendChild(nuevoDiv);

botonSubmit.addEventListener('click', (event) => {
  event.preventDefault(); // Previene la recarga del formulario
  console.log('Formulario enviado sin recargar página');
});
```

6. Asincronía: Promesas y `async/await`

```
// Fetch API con async/await y try/catch/finally
async function cargarDatosAPI(url) {
  try {
    const respuesta = await fetch(url);

    if (!respuesta.ok) {
      throw new Error(`Error HTTP: ${respuesta.status}`);
    }

    const datos = await respuesta.json();
    return datos;
  } catch (error) {
```

```

        console.error('Fallo en la petición:', error.message);
        throw error;
    } finally {
        console.log('Petición HTTP completada.');
```

```
    }
```

```
}
```

```
// Ejecución en paralelo con Promise.all
```

```
async function cargarTodo() {
```

```
    const [usuarios, productos] = await Promise.all([
```

```
        cargarDatosAPI('/api/usuarios'),
```

```
        cargarDatosAPI('/api/productos')
```

```
    ]);
```

```
    console.log(usuarios, productos);
```

```
}
```

7. Clases y Programación Orientada a Objetos (ES6+)

```
class Empleado {
    // Campo privado (ES2022)
    #salarioBase;

    constructor(nombre, salario) {
        this.nombre = nombre;
        this.#salarioBase = salario;
    }

    // Getter
    get salario() {
        return this.#salarioBase;
    }

    // Método estático
    static crearEmpleadoAnonimo() {
        return new Empleado('Sin Nombre', 0);
    }
}

class Gerente extends Empleado {
    constructor(nombre, salario, departamento) {
        super(nombre, salario);
        this.departamento = departamento;
    }
}
```

```
}  
}
```

8. Consejos Prácticos

■ **Consejo Práctico (Igualdad Estricta):** Use siempre `===` y `!==` en lugar de `==` y `!=`. La igualdad estricta `===` compara valor Y tipo sin realizar conversiones de tipo implícitas (*Type Coercion*).

■ **Atención (Funciones Mutantes de Array):** Métodos como `push`, `pop`, `sort`, `splice` mutan el arreglo original. Prefiera métodos inmutables como `map`, `filter`, `concat` o sintaxis `spread [...arr]`.